

with the spare capacity (target capacity - current capacity) on the receivers as the bin size and the VM migration list as the balls. *iFFD* has the nice property that it starts with an initial allocation, and then incrementally finds a set of migrations that will help it reach the target utilization for each server. Hence, the packing algorithm migrates VMs only to the extent required to reach the target utilization for each server.

We use *iFFD* to design a power-minimizing placement algorithm that includes history, and is aptly named as *min Power Placement algorithm with History mPPH*. It works identically as *mPP* in the first phase. For the second phase, it invokes *iFFD* instead of *FFD*, thus selecting a placement that takes the earlier placement as a starting point. *mPPH* algorithm tries to minimize migrations by migrating as few VMs as possible, while moving to the new optimal target utilization for the servers. However, even though it is migration aware, it does not compromise on the power minimization aspect of the algorithm. Hence, if the target utilization for servers change significantly in a time-window, *mPPH* still resorts to large scale migrations to minimize power cost.

We next propose an algorithm *PMaP* that takes a balanced view of both power and migration cost, and aims to find an allocation that minimizes the sum of the total (power + migration) cost. *pMaP* (Fig. 6(b)) continually finds a new placement for VMs in a fashion that minimizes power, while taking the migration cost into account. The algorithm is based on the fundamental observation that all the migrations that take us from an old power-minimizing placement to a new power-minimizing placement may not be optimizing the power-migration cost tradeoff. Hence, the algorithm first invokes any power-minimizing placement algorithm (*mPPH* is our choice) and gets a new power-minimizing placement. It then computes the difference between the two placements (the set of migrations that will change the old placement to the new placement) and determines a subset to select. The selection process is based on sorting all the migrations based on their incremental decrease in power per unit migration cost. We note that a set of multiple migrations may be atomic and have to be separately considered en masse as well, while estimating their incremental decrease in power per unit migration cost. We then select the most profitable migration if the power savings due to migration is higher than the migration cost. We repeat the above procedure till no migrations exist that optimize the power-migration cost tradeoff. We prove the following *Local Optimality* property for *PMaP* along the same lines as Theorem 1.

Lemma 1. *If pMaP at any given time has a placement P, then the next migration selected by P achieves the highest power-migration cost tradeoff. Hence, every iteration that selects the next migration in pMaP is locally optimal.*

5 pMapper Implementation and Experimental Validation

We now present our implementation of *pMapper* and an experimental study to demonstrate its effectiveness.